

To enable h265 video streaming at webrtc do the following

a) Take a camera having h265 stream

b) change the branch to git checkout h265_cam_support

c) git clone https://bitbucket.org/multicoreware/x265_git.git

```
cd x265/build/linux
```

```
./make-Makefiles.bash
```

```
make -j8
```

```
make install
```

```
cd /workspace/ffmpeg
```

```
./configure --pkg-config-flags="--static" --libdir=/usr/local/lib --disable-shared --  
enable-static --enable-gpl --enable-pthreads --enable-nonfree --enable-libfdk-aac --  
enable-libx265 --enable-libx264 --enable-filters --enable-runtime-cpudetect --  
disable-lzma
```

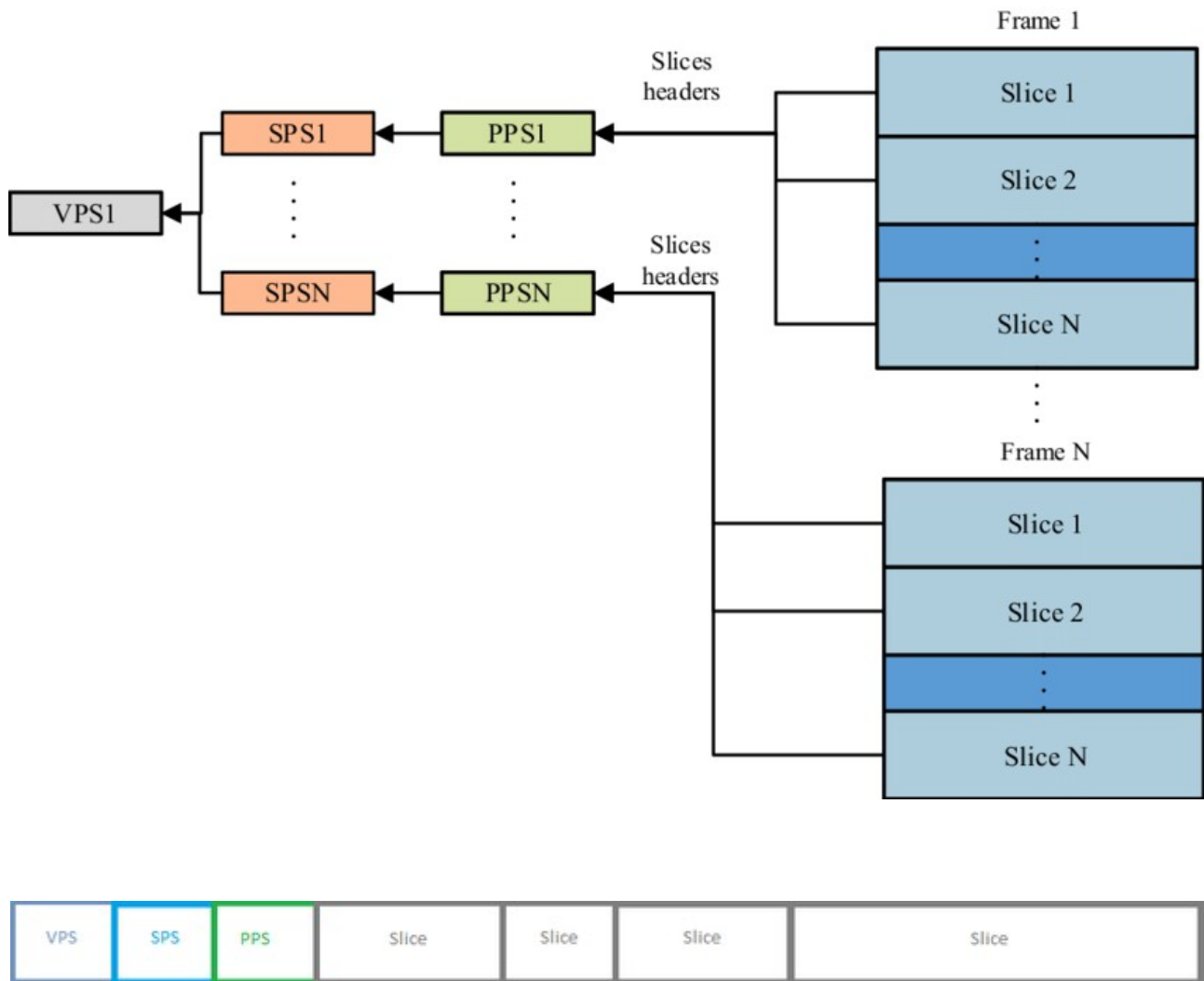
d) cd /workspace/mediaserver/src/broadcast/main

```
make clean
```

```
make -j8
```

```
systemctl restart webrtc
```

Add VPS



HEVC bitstream is an ordered sequence of the syntax elements. Each syntax element is placed into a logical packet called a NAL (network abstraction layer) Unit. There are 64 different NAL Unit types. They can be grouped into 10 classes:

- 1.VPS - Video parameter set
- 2.SPS - Sequence parameter set
- 3.PPS - Picture parameter set
- 4.Slice (different types)
- 5.AUD - Access unit delimiter
- 6.EOS - End of sequence
- 7.EOB - End of bitstream
- 8.FD - Filler data
- 9.SEI - Supplemental enhancement information
- 10.Reserved and unspecified

VPS, SPS and PPS contain general video parameters. They provide a robust mechanism for conveying data that are essential to the decoding process. They can be either a part of bitstream or can be stored separately.

Slice NAL unit contains data from encoded video frame. It can contain full frame or its part. Each slice can be decoded independently, that is, without using information from any other slice. Thereby, slices can be used as a tool to support parallel encoding/decoding.

There are following slice types:

- 1.I-slice - slice with only intra prediction
- 2.P-slice - slice with inter prediction from one I or P slices
- 3.B-slice - slice with inter prediction from two I or P slices

There is one special slice type called IDR-slice. There are no references from slices after IDR-slice to slices before it.

AUD can be used for signaling about start of video frame.

FD can be used for bitrate smoothing.

SEI provides support for different types of metadata. It includes picture timing, color space information, etc.

Information about FPS is placed in

`sps:vui:num_units_in_tick,sps:vui:time_scale`
and `vps:num_units_in_tick,vps:time_scale`. FPS in SPS and VPS can be present together, one of them or neither.

$$\text{FPS} = \text{sps:vui:time_scale} / \text{sps:vui:num_units_in_tick} = \text{vps:time_scale} / \text{vps:num_units_in_tick}$$

Size of picture is stored in

`sps:pic_width_in_luma_samples` and `sps:pic_height_in_luma_samples`.

Aspect ratio can be present as predefined constant or as a pair of horizontal and vertical values. Aspect ratio value is placed into `sps::vui::aspect_ratio_idc`.

If `sps::vui::aspect_ratio_idc==EXTENDED_SAR` then `sar_width` and `sar_height` are used.

Enable H265 at Live.cpp

```
void FrameSink::afterGettingHeader(unsigned frameSize, unsigned
numTruncatedBytes, struct timeval presentationTime, unsigned
/*durationInMicroseconds*/) {
    // We've just received a frame of data. (Optionally) print out information about it:
#ifdef DEBUG_PRINT_EACH_RECEIVED_FRAME
    if (fStreamId != NULL) envir() << "Stream \"" << fStreamId << "\"; ";
    envir() << fSubsession.mediumName() << "/" << fSubsession.codecName() << ":\"
tReceived " << frameSize << " bytes";
    if (numTruncatedBytes > 0) envir() << " (with " << numTruncatedBytes << " bytes
truncated)";
    char uSecsStr[6+1]; // used to output the 'microseconds' part of the presentation
time
    sprintf(uSecsStr, "%06u", (unsigned)presentationTime.tv_usec);
    envir() << ".\tPresentation time: " << (int)presentationTime.tv_sec << "." <<
uSecsStr;
    if (fSubsession.rtpSource() != NULL && !fSubsession.rtpSource()-
>hasBeenSynchronizedUsingRTCP()) {
        envir() << " !"; // mark the debugging output to indicate that this presentation
time is not RTCP-synchronized
    }
#endif
#ifdef DEBUG_PRINT_NPT
    envir() << "\tNPT: " << fSubsession.getNormalPlayTime(presentationTime);
#endif
    envir() << "\n";
#endif

    basicframe.copyBuf(fReceiveBuffer, frameSize );
    // mstimestamp=presentationTime.tv_sec*1000+presentationTime.tv_usec/1000;
    // std::cout << "afterGettingFrame: mstimestamp=" << mstimestamp <<std::endl;

    basicframe.mstimestamp=(presentationTime.tv_sec*1000+presentationTime.tv_use
c/1000);
    basicframe.fillPars();
    // std::cout << "afterGettingFrame: " << basicframe ;

    // basicframe.payload.resize(checkBufferSize(frameSize)); // set correct frame size ..
now information about the packet length goes into the filter chain

    scs.setFrame(); // flag that indicates that we got a frame
```

```

    if (basicframe.h265_pars.slice_type == H265SliceType::SPS_NUT ||
basicframe.h265_pars.slice_type == H265SliceType::PPS_NUT ||
basicframe.h265_pars.slice_type == H265SliceType::VPS_NUT) //AUD Delimiter
    {

        unsigned num_units_in_tick, time_scale;

        // analyze_seq_parameter_set_data(buffer,sz, num_units_in_tick, time_scale);

        if( !foundspcs && basicframe.h265_pars.slice_type ==
H265SliceType::SPS_NUT )
        {

            obj.analyze_seq_parameter_set_data(fReceiveBuffer , frameSize,
num_units_in_tick, time_scale);

            if( !obj.fps)
            {
                if(fpsFromSDP)
                    obj.fps = fpsFromSDP;
                else
                    obj.fps = 5;
            }

            #if FPSONE
            basicframe.copyBuf(fReceiveBuffer, frameSize );
            #endif

            fps = obj.fps ;
            height = obj.height;
            width = obj.width;

            basicframe.fps = obj.fps ;
            basicframe.height = obj.height;
            basicframe.width = obj.width;

            SInfo << " Got SPS fps " << fps << " width " << width << " height " <<
height ;

            //info->run(&basicframe);
            if(fragmp4_muxer)
                fragmp4_muxer->run(&basicframe); // starts the frame filter chain

            if(mvExtractor)

```

```

        mvExtractor->run(&basicframe); // starts the frame filter chain

        basicframe.payload.resize(basicframe.payload.capacity());

        foundsps = true;

    }

    if( !foundpps && (basicframe.h265_pars.slice_type == H265SliceType::PPS_NUT
|| basicframe.h265_pars.slice_type == H265SliceType::VPS_NUT) )
    {

        SInfo << " Got PPS fps ";

        //info->run(&basicframe);
        if(fragmp4_muxer)
            fragmp4_muxer->run(&basicframe); // starts the frame filter chain

        if(mvExtractor)
            mvExtractor->run(&basicframe); // starts the frame filter chain

        basicframe.payload.resize(basicframe.payload.capacity());

        foundpps = true;

    }

}

// std::cerr << "BufferSource: IN0: " << basicframe ;
//info->run(&basicframe);
// fragmp4_muxer->run(&basicframe); // starts the frame filter chain

// if (numTruncatedBytes>0) { // time to grow the buffer..
//     SDebug << "FrameSink : growing reserved size to "<< target_size << " bytes" ;
//     setReceiveBuffer(target_size);
// }

// basicframe.payload.resize(basicframe.payload.capacity()); // recovers maximum
size .. must set maximum size before letting live555 to write into the memory area

// Then continue, to request the next frame of data:
if (on) {continuePlaying();}
}

```